

Guía Completa de Colecciones y Estructuras de Datos en Java

Material de apoyo extendido para estudiantes

En este documento exploraremos a fondo las principales estructuras de datos proporcionadas por la API de Java (`Stack` , `Queue` , `LinkedList` y `PriorityQueue`). Revisaremos sus diferencias teóricas, operaciones principales (con sus equivalentes seguros) y múltiples ejemplos prácticos de uso.

1. Stack (Pila)

Una pila es una estructura **LIFO** (Last In, First Out / Último en entrar, Primero en salir). Piensa en ella como una pila de libros: el último libro que pones encima es el primero que debes quitar para acceder a los de abajo.

- **¿Cuándo usarla?**
 - Historiales de navegación (botón "Atrás").
 - Evaluación de expresiones matemáticas y validación de sintaxis (paréntesis balanceados).
 - Algoritmos de Backtracking y recorridos de grafos (DFS - Búsqueda en Profundidad).
- **Operaciones Principales:**
 - `push(E item)` : Agrega un elemento a la cima de la pila.
 - `pop()` : Remueve y devuelve el elemento de la cima. Lanza `EmptyStackException` si está vacía.
 - `peek()` : Devuelve el elemento de la cima sin removerlo.
 - `isEmpty()` : Verifica si la pila está vacía.
 - `search(Object o)` : Busca un objeto y devuelve su distancia desde la cima (1 es el tope). Retorna `-1` si no existe.

Ejemplos Prácticos

Ejemplo 1: Invertir una palabra

```
import java.util.Stack;

public class InvertirPalabra {
    public static void main(String[] args) {
        String palabra = "JAVA";
        Stack<Character> pila = new Stack<>();

        // Apilamos cada letra
        for(char c : palabra.toCharArray()) {
            pila.push(c);
        }

        // Desapilamos (saldrán en orden inverso)
        System.out.print("Palabra invertida: ");
        while(!pila.isEmpty()) {
            System.out.print(pila.pop()); // Imprime: AVAJ
        }
    }
}
```

```

    }
}
}

```

Ejemplo 2: Validación rápida de historial

```

Stack<String> historial = new Stack<>();
historial.push("google.com");
historial.push("github.com");
historial.push("stackoverflow.com");

System.out.println("Página actual: " + historial.peek()); // stackoverflow.com
System.out.println("Distancia a google.com: " + historial.search("google.com")); //
3

```

2. Queue (Cola)

Una cola es una estructura **FIFO** (First In, First Out / Primero en entrar, Primero en salir). Funciona exactamente como una fila en un banco: el primero en llegar es el primero en ser atendido.

- **Nota importante:** En Java, `Queue` es una **Interfaz**. No puedes instanciarla directamente (`new Queue<>()`). Debes usar clases que la implementen, como `LinkedList` o `PriorityQueue` .
- **¿Cuándo usarla?**
 - Colas de impresión, tareas asíncronas o buffer de datos.
 - Algoritmos de Búsqueda a lo Ancho en grafos (BFS).
 - Procesamiento en orden estricto de llegada.
- **Operaciones Principales (Excepciones vs Valores Especiales):** La API de Java provee dos juegos de métodos para colas. Uno lanza excepciones si falla, el otro devuelve `false` o `null` .
 - **Insertar:** `add(E e)` (Lanza excepción) vs `offer(E e)` (Retorna `false` si no hay espacio).
 - **Remover:** `remove()` (Lanza excepción si está vacía) vs `poll()` (Retorna `null` si está vacía).
 - **Ver Frente:** `element()` (Lanza excepción) vs `peek()` (Retorna `null`).

Ejemplos Prácticos

Ejemplo 1: Fila de atención básica

```

import java.util.Queue;
import java.util.LinkedList;

Queue<String> filaBanco = new LinkedList<>();

// Usamos offer porque es más seguro (no lanza excepción si la cola tuviera un
límite)
filaBanco.offer("Juan");
filaBanco.offer("Ana");
filaBanco.offer("Pedro");

```

```
System.out.println("Siguiente en ser atendido: " + filaBanco.peek()); // Juan

while(!filaBanco.isEmpty()) {
    System.out.println("Atendiendo a: " + filaBanco.poll());
}
// Salida: Juan, luego Ana, luego Pedro
```

3. LinkedList (Lista Doblemente Enlazada y Deque)

`LinkedList` es una "navaja suiza" en Java. Implementa las interfaces `List`, `Queue`, y `Deque` (Double Ended Queue). Guarda elementos en "nodos" que conocen al elemento anterior y al siguiente.

- **¿Cuándo usarla?**
 - Cuando necesitas insertar o eliminar elementos frecuentemente en los extremos (al inicio o al final), lo cual es rápido $O(1)$.
 - Como implementación estándar de una Cola (`Queue`).
 - Evitarla si necesitas acceder aleatoriamente por índices (como `lista.get(5000)`), ahí es mejor `ArrayList` .
- **Operaciones Principales como Lista y Deque:**
 - `addFirst(E e)` / `addLast(E e)` : Inserción en extremos.
 - `removeFirst()` / `removeLast()` : Eliminación en extremos.
 - `getFirst()` / `getLast()` : Obtener sin eliminar.
 - Operaciones de lista estándar: `add(index, e)` , `remove(Object o)` .

Ejemplos Prácticos

Ejemplo 1: Uso como Cola de Doble Extremo (Deque)

```
import java.util.LinkedList;

LinkedList<String> tren = new LinkedList<>();
tren.add("Vagón 1"); // Comportamiento normal (añade al final)

// Manipulación por extremos
tren.addFirst("Locomotora");
tren.addLast("Vagón Cola");

System.out.println("Inicio del tren: " + tren.getFirst()); // Locomotora
System.out.println("Fin del tren: " + tren.getLast()); // Vagón Cola

// Removiendo de ambos extremos
tren.removeFirst();
tren.removeLast();
System.out.println(tren); // [Vagón 1]
```

4. PriorityQueue (Cola de Prioridad)

Es una cola especial donde **los elementos NO salen en orden de llegada**, sino **por orden de prioridad**. Utiliza internamente una estructura de "Min-Heap" (Montículo binario).

- **¿Cuándo usarla?**
 - Sistema de emergencias (Triage de hospital).
 - Algoritmos de grafos basados en pesos (Ej. Dijkstra, Prim).
 - Para mantener siempre accesible el elemento "mínimo" o "máximo" de un flujo de datos dinámico.
- **Operaciones Principales (Las mismas de Queue, $O(\log n)$ para inserción/remoción):**
 - `offer(E e)` : Inserta reordenando el árbol internamente.
 - `poll()` : Remueve y retorna la máxima prioridad (por defecto el valor "más pequeño").
 - `peek()` : Observa la mayor prioridad en $O(1)$.

Ejemplos Prácticos

Ejemplo 1: Orden natural (Menor a Mayor)

```
import java.util.PriorityQueue;

PriorityQueue<Integer> turnos = new PriorityQueue<>();
turnos.offer(40);
turnos.offer(10); // Llega después, pero es un número menor
turnos.offer(25);

// Saldrán en orden ascendente, sin importar cuándo llegaron
while(!turnos.isEmpty()) {
    System.out.println("Turno llamado: " + turnos.poll());
}
// Imprime: 10, luego 25, luego 40
```

Ejemplo 2: Usar un orden personalizado (Mayor a Menor)

```
import java.util.PriorityQueue;
import java.util.Collections;

// Le pasamos un Comparator para invertir el orden natural (Max-Heap)
PriorityQueue<Integer> sueldosMayores = new PriorityQueue<>
(Collections.reverseOrder());
sueldosMayores.offer(1500);
sueldosMayores.offer(3000);
sueldosMayores.offer(2000);

System.out.println("Mayor sueldo: " + sueldosMayores.poll()); // Imprime 3000
```